

SEPHORA COSMETIC REVIEWS: AI-Powered Review Rating System

1. Dataset Used : Sephora Products and Skincare Reviews

This dataset contains: information about all beauty products (over 8,000) from the Sephora online store, including product and brand names, prices, ingredients, ratings, and all features. user reviews (about 1 million on over 2,000 products) of all products from the Skincare category, including user appearances, and review ratings by other users.

1.1 Objective

Build an automated machine learning system that predicts the star rating (1-5 stars) of a cosmetic product review based purely on the text content. This is a supervised, multi-class text classification problem.

1.2 Key Difference from Balanced Pipeline

Aspect	Balanced (Step 1A)	Imbalanced (Step 1B)
Sampling	20,000 per class (equal)	Custom % per class (skewed)
Total Rows	100,000	100,000
★1 share	20%	5% (5,000 rows)
★2 share	20%	10% (10,000 rows)
★3 share	20%	15% (15,000 rows)
★4 share	20%	25% (25,000 rows)
★5 share	20%	45% (45,000 rows)
Purpose	Fair per-class evaluation	Realistic distribution study

2. Libraries and Environment

2.1 Imports Used

Library	Module / Component	Purpose
pandas	pandas	Data loading, cleaning, grouping, feature engineering
nltk	nltk.corpus.stopwords	English stopwords list
nltk	nltk.stem.WordNetLemmatizer	Word lemmatization (root form extraction)
nltk	nltk.download()	Downloads stopwords, wordnet, omw-1.4 data
re	re (standard library)	Regular expressions for URL/HTML/special char removal
string	string (standard library)	string.punctuation for punctuation stripping
matplotlib	matplotlib.pyplot	All chart rendering
seaborn	seaborn	Statistical plots: countplot, histplot, boxplot, barplot
sklearn	model_selection.train_test_split	Stratified 80/20 split
sklearn	feature_extraction.text.TfidfVectorizer	TF-IDF vectorization

2.2 NLTK Downloads

```
import nltk
nltk.download('stopwords') # English stopwords list
nltk.download('wordnet') # Lemmatization vocabulary
```

3. Data Loading and Initial Inspection

`df.head(5)`

Confirmed column names, data types, and that review_text contains natural English. Showed is_recommended, helpfulness, review_title, skin_tone already have NaN in the first rows.

df.shape -> (602130, 19)

df.columns.tolist()

Listed all 19 column names. Confirmed review_text and rating are present.

df.dtypes

Confirmed rating is int64 (no conversion needed), 5 int columns, 3 float columns, 11 object/string columns.

df.isnull().sum() - Missing Values Audit

Column	Null Count	% Missing	Action
review_text	999	0.17%	Dropped rows (critical)
review_title	167,011	27.7%	Column kept
is_recommended	117,486	19.5%	Column kept
helpfulness	331,832	55.1%	Column kept
skin_tone	106,056	17.6%	Column kept
eye_color	138,488	23.0%	Column dropped
skin_type	74,683	12.4%	Column kept
hair_color	141,081	23.4%	Column dropped
All other columns	0	0%	No action

Initial Rating Distribution

Rating	Count	Percentage
★ 1	32,903	5.47%
★ 2	29,253	4.86%
★ 3	44,018	7.31%
★ 4	106,956	17.76%
★ 5	389,000	64.60%

4. Data Cleaning

Identified missing values across all columns. Removed rows with missing review_text values (999 records).

Verified that the rating column contained no missing values and only valid ratings from 1 to 5.

Missing Values:

Unnamed: 0	0
author_id	0
rating	0
is_recommended	117486
helpfulness	331832
total_feedback_count	0
total_neg_feedback_count	0
total_pos_feedback_count	0
submission_time	0
review_text	999
review_title	167011
skin_tone	106056
eye_color	138488
skin_type	74683
hair_color	141081
product_id	0
product_name	0
brand_name	0
price_usd	0

4.1 Remove Null Review Text

999 rows where review_text was NaN were dropped - these have no input for the model.

```
df = df.dropna(subset=['review_text'])  
# 602,130 -> 601,131 rows
```

A follow-up check confirmed 0 empty string reviews after whitespace stripping.

4.2 Verify Rating Values

```
df['rating'].unique() -> [5 1 4 2 3]
```

All values are exactly integers 1-5, no out-of-range values

4.3 Duplicate Analysis

Checked for exact duplicate records and removed them where applicable. Analyzed conflicting reviews (same review text with different ratings). Only about 100 conflicting review texts were found. These were retained because they represent real-world variation in user opinions and formed a negligible portion of the dataset.

Duplicate review texts (same text, different rows)

```
df['review_text'].duplicated().sum()
```

Duplicate Review Texts: 92,664

92,664 rows share text with at least one other row. Not dropped because the same review text may legitimately appear for different products.

Conflicting reviews (same text, different ratings)

```
conflicting = df.groupby('review_text')['rating'].nunique()  
conflicting = conflicting[conflicting > 1]
```

Duplicate review text analysis identified 100 conflicting review texts associated with multiple rating values. Since these represented less than 0.02% of the dataset and may reflect genuine differences in user perception, they were retained in the final dataset.

4.4 Column Selection

From 19 original columns, 9 were kept:

Shape: (601,131, 9)

Status	Columns
Kept	review_text, rating, review_title, is_recommended, helpfulness, skin_type, skin_tone, product_name, brand_name

Dropped	Unnamed:0, author_id, total_feedback_count, total_neg_feedback_count, total_pos_feedback_count, submission_time, eye_color, hair_color, price_usd
---------	---

5. Text Preprocessing

5.1 NLTK Setup

```
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
stop_words = set(stopwords.words('english')) # 179 stopwords
lemmatizer = WordNetLemmatizer()
```

5.2 The preprocess_text() Function

A single reusable function applies 8 sequential cleaning operations to each review:

```
def preprocess_text(text):
    text = str(text).lower() # 1. Lowercase
    text = re.sub(r'http\S+|www\S+|https\S+', '', text) # 2. Remove URLs
    text = re.sub(r'<.*?>', '', text) # 3. Strip HTML
    text = text.translate( # 4. Remove punctuation
        str.maketrans("", "", string.punctuation))
    text = re.sub(r'^a-zA-Z0-9\s', '', text) # 5. Special chars
    text = re.sub(r'\s+', ' ', text).strip() # 6. Collapse spaces
    words = text.split()
    words = [lemmatizer.lemmatize(word) # 8. Lemmatize
             for word in words
             if word not in stop_words] # 7. Remove stopwords
    return ' '.join(words)
```

Stage	Operation	Before	After
1	Lowercase	'Great Product!'	'great product!'
2	Remove URLs	'Buy at	'Buy at '

		https://sephora.com	
3	Strip HTML	'Rated 5 stars'	'Rated 5 stars'
4	Remove punctuation	'great, perfect!'	'great perfect'
5	Special characters	'100% pure'	'100 pure'
6	Collapse whitespace	'great skin care'	'great skin care'
7	Remove stopwords	['i','really','love','this']	['really','love']
8	Lemmatize	['hydrating','loved']	['hydrate','love']

Stopwords :

['a', 'about', 'above', 'after', 'again', 'against', 'ain', 'all', 'am', 'an', 'and', 'any', 'are', 'aren', 'aren't', 'as', 'at', 'be', 'because', 'been', 'before', 'being', 'below', 'between', 'both', 'but', 'by', 'can', 'couldn', 'couldn't', 'd', 'did', 'didn', 'didn't', 'do', 'does', 'doesn', 'doesn't', 'doing', 'don', 'don't', 'down', 'during', 'each', 'few', 'for', 'from', 'further', 'had', 'hadn', 'hadn't', 'has', 'hasn', 'hasn't', 'have', 'haven', 'haven't', 'having', 'he', 'he'd', 'he'll', 'he's', 'her', 'here', 'hers', 'herself', 'him', 'himself', 'his', 'how', 'i', 'i'd', 'i'll', 'i'm', 'i've', 'if', 'in', 'into', 'is', 'isn', 'isn't', 'it', 'it'd', 'it'll', 'it's', 'its', 'itself', 'just', 'll', 'm', 'ma', 'me', 'mightn', 'mightn't', 'more', 'most', 'mustn', 'mustn't', 'my', 'myself', 'needn', 'needn't', 'no', 'nor', 'not', 'now', 'o', 'of', 'off', 'on', 'once', 'only', 'or', 'other', 'our', 'ours', 'ourselves', 'out', 'over', 'own', 're', 's', 'same', 'shan', 'shan't', 'she', 'she'd', 'she'll', 'she's', 'should', 'should've', 'shouldn', 'shouldn't', 'so', 'some', 'such', 't', 'than', 'that', 'that'll', 'the', 'their', 'theirs', 'them', 'themselves', 'then', 'there', 'these', 'they', 'they'd', 'they'll', 'they're', 'they've', 'this', 'those', 'through', 'to', 'too', 'under', 'until', 'up', 've', 'very', 'was', 'wasn', 'wasn't', 'we', 'we'd', 'we'll', 'we're', 'we've', 'were', 'weren', 'weren't', 'what', 'when', 'where', 'which', 'while', 'who', 'whom', 'why', 'will', 'with', 'won', 'won't', 'wouldn', 'wouldn't', 'y', 'you', 'you'd', 'you'll', 'you're', 'you've', 'your', 'yours', 'yourself', 'yourselves']

5.3 Before / After Examples from Notebook

ORIGINAL:

I use this with the Nudestix "Citrus Clean Balm & Make-Up Melt" to double cleanse and it has completely changed my skin (for the better). The make-up melt is oil based and removes all of your makeup super easily.

CLEANED:

use nudestix citrus clean balm makeup melt double cleanse completely changed skin better makeup melt oil based remove makeup super easily

ORIGINAL:

I bought this lip mask after reading the reviews and the hype. Unfortunately, it did not meet my expectations as vaseline petroleum jelly works way better for me.

CLEANED:

bought lip mask reading review hype unfortunately meet expectation vaseline petroleum jelly work way better

6. Review Length Filtering

6.1 Word Count Statistics

```
df['word_count'] = df['clean_review'].str.split().str.len()
```

Statistic	Value	Interpretation
count	601,044	Reviews after null removal
mean	30.14	Average: ~30 cleaned words per review
std	21.73	High variance in review length
min	1	Shortest reviews: single word
25%	16	Bottom quarter: 16 words or fewer
50% (median)	25	Half of reviews have 25 words or fewer
75%	38	Top quarter: 38+ words
max	525	Longest review outlier

6.2 Filter Applied

```
df = df[(df['word_count'] >= 3) & (df['word_count'] <= 300)]
```


Shape after: (600,342, 16)

Threshold	Value	Rationale
Minimum	3 words	Reviews with 1-2 cleaned words carry no meaningful sentiment signal
Maximum	No upper cap	

7. Feature Engineering

Five numerical features were derived from the review text and stored as new DataFrame columns:

```
df['word_count'] = df['clean_review'].str.split().str.len()
df['char_count'] = df['clean_review'].str.len()
df['review_length'] = df['clean_review'].str.len() # same as char_count
df['avg_word_length'] = (df['char_count'] / df['word_count']).round(2)
df['exclamation_count'] = df['review_text'].astype(str).str.count('!')
df['question_count'] = df['review_text'].astype(str).str.count(r'\?')
```

Feature	Source	Mean	Std	Min	Max
word_count	clean_review	30.16	21.65	3	295
char_count	clean_review	195.63	142.62	8	1,949
review_length	clean_review	195.63	142.62	8	1,949
avg_word_length	clean_review	6.46	0.57	2.00	18.38
exclamation_count	review_text	--	--	0	varies
question_count	review_text	--	--	0	varies

8. Class Distribution Analysis

After all cleaning and filtering, the final class distribution before balancing:

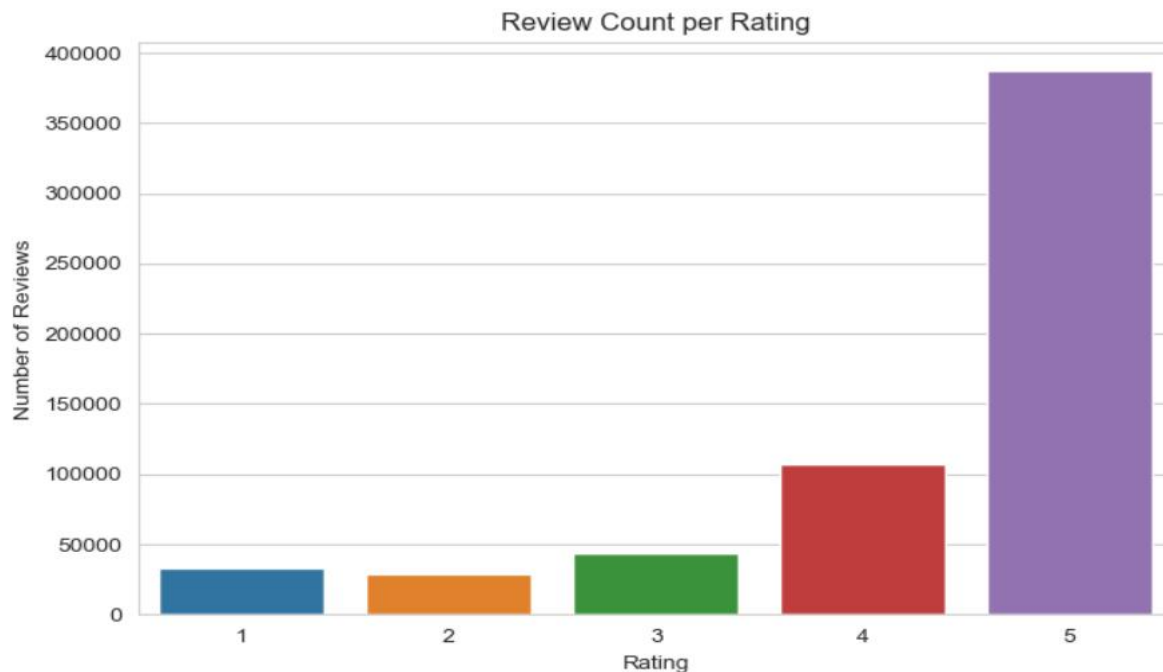
Rating	Count	Percentage	Observation
★ 1	32,789	5.46%	Negative reviews - minority class

★ 2	29,195	4.86%	Below average - rarest class
★ 3	43,950	7.32%	Neutral reviews
★ 4	106,747	17.78%	Good reviews
★ 5	387,661	64.57%	Dominant class (nearly 2 in 3 reviews)
Total	600,342	100%	--

9. Data Visualization

Original Rating Distribution — Bar Chart

`sns.countplot` showing raw number of reviews per class. Clearly shows the 5-star dominance and motivates class balancing.



10. Imbalanced Dataset Creation

10.1 Strategy

Rather than sampling uniformly (20,000 per class) as in Step 1A, this pipeline constructs an intentionally **imbalanced dataset of 100,000 reviews** that approximates a realistic cosmetics review distribution. Higher-rated products receive disproportionately more reviews, mirroring what a deployed model would encounter in production.

9.2 Sampling Strategy

```
total_reviews = 100000
# Imbalanced distribution weights
distribution = {
  1: 0.05, # 5,000 reviews
  2: 0.10, # 10,000 reviews
  3: 0.15, # 15,000 reviews
  4: 0.25, # 25,000 reviews
  5: 0.45, # 45,000 reviews
}
imbalanced_df = pd.concat([
  df[df['rating'] == rating].sample(
    n=int(total_reviews * percentage),
    random_state=42
  )
  for rating, percentage in distribution.items()
])
# Shuffle all rows
imbalanced_df = imbalanced_df.sample(frac=1,
  random_state=42).reset_index(drop=True)
```

Result

Rating	Available	Sampled	Final %	Approx.Ratio
★ 1	32,789	5,000	5.0%	1x(Minority)
★ 2	29,195	10,000	10.0%	2x
★ 3	43,950	15,000	15.0%	3x

★ 4	106,747	25,000	25.0%	5x
★ 5	387,661	45,000	45.0%	9x(Majority)
Total	600,342	100,000	100%	--

10.3 Shuffle and Save

```
# Shuffle all 100K rows randomly
balanced_df = balanced_df.sample(frac=1,
random_state=42).reset_index(drop=True)

# Save to disk
balanced_df.to_csv('sephora_cleaned_dataset.csv', index=False)
# Output: Balanced dataset saved.
```

11. Stratified Train-Test Split

11.1 Defining X and y

```
X = imbalanced_df['clean_review'] # 100,000 cleaned review strings
y = imbalanced_df['rating']       # 100,000 integer labels (1-5)
```

11.2 Split Parameters

```
from sklearn.model_selection import train_test_split

X_train_text, X_test_text, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20, # 20% held out for testing
    random_state=42, # reproducible split
    stratify=y      # preserve class proportions in both sets
)
```

11.3 Results

Split	Total Rows	★1	★2	★3	★4	★5
Training (X_train, y_train)	80,000	4,000	8,000	12,000	20,000	36,000
Test (X_test, y_test)	20,000	1,000	2,000	3,000	5,000	9,000

Split	Shape
X_train	(80,000,)
X_test	(20,000,)
y_train	(80,000,)
y_test	(20,000,)

12. TF-IDF Vectorization

12.1 What is TF-IDF?

TF-IDF (Term Frequency–Inverse Document Frequency) converts text into numerical feature vectors processable by machine learning algorithms. Each review becomes a 10,000-element sparse vector representing the importance of words it contains.

Formula	Expression	Meaning
TF(t,d)	$\text{count}(t \text{ in } d) / \text{total_terms}(d)$	How often term t appears in review d
IDF(t)	$\log(N / \text{DF}(t))$	How rare term t is across all reviews
TF-IDF	$\text{TF}(t,d) \times \text{IDF}(t)$	Combined importance score

12.2 Vectorizer Configuration

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
tfidf = TfidfVectorizer(
    max_features=10000, # vocabulary capped at 10,000 terms
    ngram_range=(1, 2), # unigrams AND bigrams included
    min_df=5           # ignore terms in fewer than 5 documents
```

)

Parameter	Value	Effect
max_features	10,000	Caps vocabulary size. Each review becomes a 10,000-element sparse vector. Controls model complexity.
ngram_range	(1,1)	Unigrams only- single words,no bigrams
min_df	Default(1)	No rare-word filter applied

12.3 Final Output Shapes

Variable	Shape	Type	Meaning
X_train	(80,000 x 10,000)	scipy sparse matrix	80K training reviews as TF-IDF vectors
X_test	(20,000 x 10,000)	scipy sparse matrix	20K test reviews, same vocabulary
y_train	(80,000,)	pandas Series (int)	Labels 1-5, 4k/8k/12k/20k/36k per class
y_test	(20,000,)	pandas Series (int)	Labels 1-5, 1k/2k/3k/5k/9k per class